

Automated Hyperparameter Scheduling for Distantly Supervised NER using Reinforcement Learning and Bayesian Optimization

Anonymous ACL submission

Abstract

Distant supervision for Named Entity Recognition (NER) produces noisy and incomplete labels that make training highly sensitive to hyperparameter choices. While prior work addresses this through label denoising, the complementary problem of *automated hyperparameter scheduling*—dynamically adjusting hyperparameters during training under weak supervision—remains underexplored. We conduct the first systematic comparison of Bayesian Optimization (BO) and Reinforcement Learning (RL) for this task, evaluating Gaussian Process Thompson Sampling (GP-TS) alongside five deep RL algorithms (TD3, SAC, TRPO, DQN, GRPO) that dynamically adjust the learning rate schedule. Using BERT-CRF and RoBERTa-CRF backbones under the BOND distant supervision framework, we benchmark on three datasets of varying noise levels (Webpage, Wikigold, CoNLL-2003) against standard schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau). We find that standard schedulers provide marginal gains over static baselines ($\pm 0.004 F_1$), while the choice of backbone architecture has a substantially larger impact: BERT-CRF attains test F_1 of 0.648 on Webpage compared to 0.552 for RoBERTa-CRF. Under 2-epoch per trial budgets, SAC achieves the strongest results among schedulers (Wikigold test F_1 of 0.500 for RoBERTa, 0.506 for BERT), while GP-TS produces more interpretable configurations at lower trial counts. A 1-epoch vs. 2-epoch per trial ablation confirms that RL methods benefit disproportionately from multi-epoch trial budgets. GP-TS best-configuration analysis reveals that noisier datasets reward more aggressive learning rates and longer warmup. Our findings establish automated scheduling as a viable, complementary strategy to label denoising for distantly supervised NER, and we release our modular scheduling framework as open-source code.

1 Introduction

Named Entity Recognition (NER) identifies and classifies entity mentions in text, but supervised approaches require large, manually annotated corpora that are expensive to produce. *Distant supervision* alleviates this bottleneck by automatically generating labels from knowledge bases or heuristic rules. The resulting annotations, however, are *incomplete* and *noisy*: coverage gaps, alignment mismatches, and ambiguity introduce errors that degrade model performance and destabilize training.

In such noisy settings, hyperparameter choices, including learning rate, weight decay, and optimizer momentum, strongly affect generalization. Static configurations fail to accommodate the non-stationarity of learning under weak supervision, motivating *automated hyperparameter scheduling*: adjusting hyperparameters during training based on model feedback.

Two paradigms address this problem. *Bayesian Optimization* (BO) methods, such as Gaussian Process Thompson Sampling (GP-TS), frame scheduling as global black-box optimization, using surrogate models and acquisition functions to balance exploration and exploitation. *Reinforcement Learning* (RL) methods treat scheduling as a sequential decision process, optimizing policies that maximize long-term validation performance. Each paradigm offers distinct trade-offs: BO is sample-efficient and interpretable but treats trials independently; RL can learn temporal policies but requires multi-epoch trajectories.

To address this gap, we present the first systematic comparison of BO and RL scheduling for distantly supervised NER. We evaluate GP-TS (probabilistic BO), TD3 (off-policy actor-critic), SAC (maximum-entropy RL), TRPO (on-policy constrained policy gradient), DQN (value-based discrete-action RL), and GRPO (Group Relative Policy Optimization), a recent PPO variant that

eliminates the critic network by estimating advantages through group-relative comparisons. We apply these methods to dynamically adjust hyperparameters during training, using BERT-CRF and RoBERTa-CRF as base models, and compare against Step Decay, Cosine Annealing, and ReduceLROnPlateau baselines across three benchmark datasets.

Our experiments on **Webpage**, **Wikigold**, and **CoNLL-2003** demonstrate the trade-offs between different scheduling strategies. On **Webpage**, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.552; standard schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau) produce similar results (0.550–0.554). For BO/RL methods under 2-epoch per-trial budgets, SAC achieves the strongest test F_1 on both **Webpage** (0.290) and **Wikigold** (0.500). BERT-CRF generalizes well, with static F_1 of 0.648 on **Webpage**. Our 1-epoch vs. 2-epoch ablation confirms that multi-epoch trial budgets improve hyperparameter selection quality. CoNLL-2003 optimizer experiments remain incomplete due to computational constraints; we report static baselines and standard scheduler results where available, and discuss this limitation.

Contributions:

1. We present the first comparative study of GP-TS and five deep RL algorithms (TD3, DQN, TRPO, SAC, GRPO) for dynamic hyperparameter scheduling in distantly supervised NER, evaluating on three benchmark datasets against three standard scheduler baselines.
2. We introduce a unified framework for modular integration of scheduling strategies into the NER training loop, supporting both BERT-CRF and RoBERTa-CRF architectures.
3. We demonstrate that standard schedulers offer modest improvements over static baselines, and that 2-epoch per-trial budgets enable RL methods to match or exceed BO approaches on Wikigold.
4. We provide an ablation study on per-trial epochs (1-epoch vs. 2-epoch), BERT vs. RoBERTa generalization, best hyperparameter configurations, and a comparison with state-of-the-art distantly supervised NER methods.

2 Related Work

2.1 Hyperparameter Optimization in Machine Learning

Hyperparameter optimization remains a core challenge in machine learning. Grid search and manual tuning scale poorly to high-dimensional spaces; random search (Bergstra and Bengio, 2012) outperforms grid search under equal budgets by sampling more broadly. Bayesian optimization models validation performance as a black-box function using a Gaussian Process surrogate. Snoek et al. (Snoek et al., 2012) demonstrated that BO matches expert-level tuning with far fewer evaluations, balancing exploration and exploitation via acquisition functions such as Expected Improvement (EI) or Upper Confidence Bound (UCB). Recent BO frameworks, including Optuna (Akiba et al., 2019), extend this paradigm with multi-objective optimization, pruning of unpromising trials, and distributed search.

Dynamic hyperparameter adaptation adjusts parameters during training rather than seeking a single optimal configuration. Population-Based Training (PBT) (Jaderberg et al., 2017) maintains a population of models with different hyperparameters, periodically exploiting (cloning top performers) and exploring (mutating hyperparameters). PBT discovers non-intuitive schedules, such as increasing learning rate late in training, that improve results across deep learning tasks. Hyperband (Li et al., 2018) accelerates hyperparameter search by adaptively allocating resources, using successive halving to prune poor configurations early. BOHB (Falkner et al., 2018) combines Bayesian optimization with Hyperband, achieving both sample efficiency and early stopping. Auto-WEKA (Feurer et al., 2015) and Auto-sklearn (Feurer et al., 2015) automate the full ML pipeline, including hyperparameter selection, but focus on static configurations rather than dynamic scheduling.

Recent work in AutoML for NLP explores neural architecture search and hyperparameter optimization for transformer models. AutoDistil (Chen et al., 2023) automates knowledge distillation for language models, while NAS-BERT (Xu et al., 2021) searches for efficient BERT architectures. OpenAutoNLU (Various, 2025b) provides end-to-end AutoML for NLU tasks including NER, using Optuna-driven hyperparameter optimization with data-aware training regime selection. Context-Aware AutoML (Various, 2025a) dynamically adapts hyperparameter search spaces

based on corpus-level meta-features, achieving improvements across 20 diverse NLP benchmarks. However, these approaches optimize architecture or static hyperparameters, not dynamic schedules during training. Our work focuses on *when* and *how* to adjust hyperparameters during the training process itself, which is particularly important under noisy supervision where the optimal schedule differs from standard training regimes.

2.2 Reinforcement Learning for Hyperparameter Tuning

Reinforcement learning methods formulate hyperparameter tuning as an MDP: the state encodes training progress, actions are hyperparameter choices, and reward reflects validation performance. Xu and Fern (Xu and Fern, 2019) used Q-learning to adjust learning rates, while Le et al. (Le et al., 2022) introduced Episodic Policy Gradient Training (EPGT) with episodic memory for dynamic hyperparameter adjustment. These approaches discover adaptive strategies that are difficult to design manually, particularly in non-stationary environments.

More recently, DeepSeekMath (Shao et al., 2024) introduced GRPO for mathematical reasoning, demonstrating that group-relative advantage estimation can replace the critic network in PPO-style training. We adapt GRPO to hyperparameter scheduling, where the reward signal (validation F_1) is well-defined and verifiable, making it a natural fit for critic-free policy optimization.

We adopt TD3 (Fujimoto et al., 2018) following the RL-for-HPO line, originally designed for continuous control. TD3 features twin Q-networks to reduce overestimation bias and delayed actor updates for stability. We adapt it to control learning rates in NER, treating training as a controllable environment. Unlike prior RL-for-HPO work that operates in low-noise settings, we evaluate RL schedulers specifically under the noisy-label conditions of distant supervision, where the reward signal itself is unreliable.

2.3 Distantly Supervised NER and Noise-Robust Training

Early distant supervision for NER (e.g., (Augenstein et al., 2017); (Peng and Dredze, 2019)) used Freebase or Wikipedia to automatically label text, yielding large but noisy datasets. BOND (Liang et al., 2020) fine-tuned BERT on noisy labels with self-training, achieving substantial gains over prior

Method	Approach	Model	Key Differences
BOND	Self-training	BERT-CRF	Denoising via self-training
RoSTER	Self-training + refinement	BERT	Label correction
SANTA	Self-training + neg. sampling	BERT	Noise-type-specific
CLIM	Class-rebalancing self-training	RoBERTa	Rebalances noisy classes
SCDL	Self-training + denoising	BERT	Confidence-driven
SALO	Adaptive label correction	BERT	Self-adaptive label correction
PBT	Population-based search	General	Static population
BOHB	BO + bandits	General	Static config space
Ours	RL + BO scheduling	RoBERTa-CRF	Dynamic scheduling

Table 1: Comparison with related methods. Our approach is complementary to denoising methods and focuses on optimizing the training schedule rather than label quality.

methods. RoSTER (Meng et al., 2021) introduced self-training with refinement to correct noisy labels in a BERT-based framework, while SANTA (Shao et al., 2023) combined self-training with negative sampling for robust distant supervision. SALO (Li et al., 2022) introduced self-adaptive label correction, achieving state-of-the-art results on noisy benchmarks. Other work (Mayhew et al., 2019) treated unlabeled tokens as uncertain tags. These BERT-based approaches demonstrate that transformer architectures, when paired with denoising strategies, significantly improve NER under distant supervision.

Our work complements these methods: rather than improving model architecture or label quality, we optimize the training process through automated hyperparameter scheduling. Our algorithms can integrate into those systems, for instance to adjust learning rates of denoising modules or control pseudo-label acceptance thresholds, but we use a standard NER model to isolate the effect of scheduling alone. To our knowledge, this is the first work exploring automated schedule optimization in distantly supervised NER.

Table 1 summarizes the key differences between our approach and prior work.

3 Methodology

We design algorithms that *automatically schedule hyperparameters*, specifically the learning rate, for NER models trained on noisy, distantly labeled datasets. Although we focus on learning rate scheduling, our framework extends to additional hyperparameters. The learning rate η_t at epoch t is dynamically adapted based on feedback from the training trajectory.

We describe two adaptive approaches: (1) a

Bayesian optimization method using Gaussian Process Thompson Sampling (GP-TS), and reinforcement learning methods including TD3, DQN, TRPO, SAC, and GRPO. GP-TS treats each adjustment as an independent experiment guided by a surrogate model, while the RL methods continuously update policies conditioned on the evolving training state.

3.1 Overview of Explored Algorithms

GP-TS and RL algorithms both operate within the training loop but differ fundamentally in methodology:

- **GP-TS (Bayesian Optimization):** Models the hyperparameter-to-performance mapping using a Gaussian Process. At each step, it samples from the GP posterior and selects the maximizing hyperparameter, enabling principled exploration-exploitation trade-offs.
- **TD3 (Reinforcement Learning):** Frames scheduling as an MDP. State s_t encodes training progress, action a_t is the learning rate, and reward r_t captures performance improvement. TD3 learns a deterministic policy $\pi_\theta(s_t) = a_t$ via twin critics Q_{ϕ_1}, Q_{ϕ_2} and a delayed actor, reducing overestimation bias.

DQN approximates the optimal action-value function $Q^*(s, a)$ using deep neural networks and the Bellman equation, operating over a discretized action space of learning rate bins.

TRPO maximizes a surrogate objective under a KL divergence constraint to prevent destabilizing policy updates, offering stable on-policy learning.

SAC optimizes both expected return and policy entropy within a maximum-entropy framework, encouraging diverse exploration in noisy landscapes.

Group Relative Policy Optimization (GRPO) is a recent variant of PPO that eliminates the need for a separate value network by estimating advantages through group-relative comparisons. For each iteration, GRPO samples G hyperparameter configurations from the current policy, evaluates each, and computes normalized advantages within the group: $\hat{A}_i = (r_i - \mu_r) / (\sigma_r + \epsilon)$, where μ_r and σ_r are the mean and standard deviation of rewards within the group. The policy is then updated using the standard PPO clipped objective on these group-relative advantages. By removing the critic, GRPO reduces model complexity and memory requirements while maintaining stable policy updates,

making it particularly suitable for hyperparameter optimization tasks with well-defined, verifiable reward signals such as validation F1 scores.

For all RL methods, training generates transitions (s_t, a_t, r_t, s_{t+1}) stored in a replay buffer. GP-TS treats each epoch as a separate trial.

3.2 Gaussian Process-Thompson Sampling (GP-TS)

GP-TS uses a Gaussian Process as a surrogate to model the performance landscape. Each input x_t is the learning rate used at epoch t (optionally paired with t itself). The observed reward r_t is the validation F1 score after epoch t .

At each step, GP-TS samples a function $f \sim P(f | \mathcal{D}_{1:t})$ from the posterior and selects the next learning rate via

$$a_{t+1} = \arg \max_{\eta} f(\eta, t + 1).$$

We use a Matérn kernel over $\log_{10}(\eta)$ and a linear trend over epoch index. A noise term accounts for observation variance. GP-TS is computationally efficient, requiring only updates to the GP posterior and an optimization over the sampled function.

Although it does not explicitly track the model's internal state, including the epoch index allows GP-TS to adapt to non-stationarity in training. This approach is conceptually simple, effective in practice, and requires no neural network training.

3.3 Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 is an off-policy actor-critic algorithm for continuous control. We define:

- **State s_t :** Includes normalized epoch index (t/T), validation F1 score, and training loss. Optionally, it may include additional features such as gradient variance.
- **Action a_t :** A real-valued learning rate selected from a constrained range, typically $[10^{-5}, 10^{-2}]$.
- **Reward r_t :** For $t < T$, we define $r_t = \text{F1}_t - \text{F1}_{t-1}$. At $t = T$, the final reward includes the overall validation F1 score.

The transition from s_t to s_{t+1} is determined by one training epoch. The actor $\pi_\theta(s_t)$ outputs the learning rate a_t , while twin critics Q_{ϕ_1}, Q_{ϕ_2} estimate the expected return. The critic loss is minimized as:

$$L(\phi_i) = \mathbb{E} [(Q_{\phi_i}(s_t, a_t) - y_t)^2],$$

$$y_t = r_t + \gamma \min_i Q_{\phi'_i}(s_{t+1}, \pi_{\theta'}(s_{t+1})),$$

where θ', ϕ' denote target networks. The actor is updated less frequently (delayed policy updates), and exploration noise is added to encourage policy diversity.

Due to limited episodes, we adopt an *online* strategy: the replay buffer collects transitions from a single training run, and multiple gradient steps are applied per epoch. Exploration noise decays over time to ensure convergence. TD3 enables the agent to learn hyperparameter scheduling policies conditioned on training dynamics. While TD3 adds moderate computation overhead, its capacity for continuous action space exploration makes it a promising candidate for adaptive scheduling in noisy NER tasks.

3.4 Other RL Algorithms: DQN, TRPO, SAC, and GRPO

For DQN, TRPO, SAC, and GRPO, we define the same state s_t , action a_t , and reward r_t as in TD3.

Deep Q-Network (DQN): DQN uses neural networks to approximate the action-value function $Q(s, a)$. It relies on the Bellman equation:

$$Q^*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} Q^*(s', a') \right].$$

DQN is suited for discrete action spaces. We define a discrete action space of $K = 6$ dimensions, each corresponding to one tunable hyperparameter. For each dimension, the agent chooses among three actions: {increase, decrease, keep}. Concretely, for the learning rate dimension, the agent selects among 20 logarithmically-spaced bins in $[10^{-6}, 10^{-4}]$, with actions mapping to adjacent bins ($\pm 5\%$ multiplicative change). The Q-network has two hidden layers of 128 units with ReLU activations and $K \times 3 = 18$ output units. An ϵ -greedy exploration strategy is used where ϵ decays from 1.0 to 0.01 over the first 50% of trials. Synchronous target network updates occur every 10 trials. DQN’s discrete action space can limit flexibility for fine-grained scheduling compared to continuous-action methods such as TD3 and SAC.

Trust Region Policy Optimization (TRPO):

TRPO is an on-policy policy gradient method that maximizes a surrogate objective under a trust region constraint to ensure stable updates:

$$\max_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right],$$

$$\text{subject to } \mathbb{E}_t [\text{KL} [\pi_{\theta_{\text{old}}}(\cdot | s_t) \| \pi_{\theta}(\cdot | s_t)]] \leq \delta.$$

This constraint, based on the Kullback-Leibler (KL) divergence, prevents drastic policy updates that may destabilize learning.

Soft Actor-Critic (SAC): SAC is an off-policy algorithm that optimizes both performance and entropy to encourage exploration. The objective is:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))],$$

where $\mathcal{H}$ denotes the entropy of the policy and α controls the entropy trade-off. SAC’s maximum-entropy framework leads to more stable behavior in noisy and uncertain environments.

Group Relative Policy Optimization (GRPO):

GRPO is a recent variant of PPO that removes the need for a separate value (critic) network. Instead of estimating advantages as $A_t = R_t - V(s_t)$, GRPO samples a group of G configurations from the current policy at each iteration, evaluates their rewards, and computes group-relative advantages:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon},$$

where μ_r and σ_r are the mean and standard deviation of rewards within the group, and ϵ is a small constant for numerical stability. The policy is then updated using the standard PPO clipped objective with these group-relative advantages. By eliminating the critic, GRPO reduces model complexity and memory requirements while maintaining stable policy updates, making it particularly suitable for hyperparameter optimization tasks with well-defined reward signals such as validation F1 scores.

Algorithm 1 presents the GRPO scheduling procedure in pseudocode.

4 Experimental Setup and Results

We evaluate our hyperparameter scheduling strategies, including GP-TS and five deep RL methods

Algorithm 1 GRPO Hyperparameter Scheduling

Require: Initial policy π_{θ_0} , group size G , clip ratio ϵ_{clip} , learning rate bounds $[\eta_{\min}, \eta_{\max}]$, number of iterations K

- 1: **for** iteration $k = 1, \dots, K$ **do**
- 2: Sample G learning rate configurations $\{\eta_i\}_{i=1}^G$ from π_{θ_k}
- 3: Train NER model for one epoch with each η_i ; collect rewards $\{r_i\}_{i=1}^G$ (validation F_1)
- 4: Compute group-relative advantages: $\hat{A}_i = (r_i - \mu_r) / (\sigma_r + \epsilon)$
- 5: Update policy via PPO clipped objective:

$$\mathcal{L}(\theta) = \mathbb{E}_i \left[\min \left(\frac{\pi_{\theta}(\eta_i | s_i)}{\pi_{\theta_{\text{old}}}(\eta_i | s_i)} \hat{A}_i, \text{clip} \left(\frac{\pi_{\theta}(\eta_i | s_i)}{\pi_{\theta_{\text{old}}}(\eta_i | s_i)}, 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}} \right) \hat{A}_i \right) \right]$$

- 6: $\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_{\theta} \mathcal{L}(\theta)$
- 7: **end for**
- 8: **return** Best learning rate configuration found across all iterations

(TD3, DQN, TRPO, SAC, GRPO), on three benchmark datasets under distantly supervised NER. We assess whether dynamic scheduling outperforms both fixed hyperparameter configurations and standard learning rate schedules in terms of NER accuracy and generalization.

4.1 Datasets and Training Baseline

We conduct experiments on three datasets widely used in distant supervision NER studies.

- **Webpage:** A small, high-noise dataset consisting of 20 web documents (e.g., homepages, academic department pages) with 783 annotated entities. Labels are generated via heuristic matching to Wikipedia entries, resulting in low recall and noisy annotations.
- **Wikigold:** A medium-scale dataset with approximately 40K tokens from Wikipedia articles, covering the same four entity types as CoNLL-2003. Distant supervision is simulated via internal hyperlink alignment and gazetteer-based labeling, yielding moderately noisy annotations.
- **CoNLL-2003:** A large, canonical NER dataset in the news domain. We discard ground-truth labels during training and instead generate noisy labels using a knowledge-base-driven alignment pipeline. Gold labels are used solely for final evaluation.

For all datasets, we adopt the RoBERTa-CRF architecture (roberta-base) as our primary NER model, with a token classification head and a

CRF decoding layer on top of the RoBERTa encoder. We also report results for BERT-CRF (bert-base-cased) to assess generalizability across transformer backbones. Both models are fine-tuned using the BOND framework (Liang et al., 2020) for distant supervision. The BOND framework uses the AdamW optimizer with a linear warmup scheduler. The static baseline uses a fixed learning rate of 1×10^{-5} and weight decay of 1×10^{-4} . The scheduling agents tune six hyperparameters: learning rate, weight decay, Adam β_1 , Adam β_2 , warmup steps, and per-GPU training batch size. In addition to the static baseline, we compare against three standard learning rate scheduler baselines:

- **Step Decay:** Reduces the learning rate by a factor of $\gamma = 0.5$ every 10 epochs.
- **Cosine Annealing** (Loshchilov and Hutter, 2017): Follows a cosine schedule over $T_{\max} = 50$ epochs with $\eta_{\min} = 1 \times 10^{-7}$.
- **ReduceLROnPlateau:** Reduces the learning rate by a factor of 0.5 after 5 epochs without improvement in validation loss, with a floor of 1×10^{-7} .

The hyperparameters tuned by the scheduling agents include: learning rate (log-uniform in $[10^{-5}, 10^{-4}]$), weight decay (log-uniform in $[10^{-5}, 10^{-3}]$), Adam β_1 (uniform in $[0.8, 0.99]$), Adam β_2 (uniform in $[0.95, 0.999]$), warmup steps (integer in $[50, 500]$), and per-GPU training batch size (integer in $[8, 32]$). To assess the benefits of adaptive scheduling, we compare these baselines with:

- **GP-TS:** Bayesian optimization over a log-uniform grid of 20 learning rate candidates, selected epoch-by-epoch.
- **TD3:** A continuous control agent that outputs a learning rate based on training state observations.
- **DQN:** A discrete-action RL agent choosing among 20 learning rate bins.
- **TRPO:** An on-policy RL method that learns a stable scheduling policy under a trust region constraint.
- **SAC:** An entropy-regularized RL agent that balances exploration and exploitation for learning rate selection.

- **GRPO**: Group Relative Policy Optimization with group size $G = 8$, which estimates advantages through group-relative comparisons without a separate value network.

4.2 Evaluation Protocol

Each method (static baseline, Step Decay, Cosine Annealing, ReduceLROnPlateau, GP-TS, TD3, DQN, TRPO, SAC, GRPO) is run under the same hardware and data conditions, with three random seeds per configuration to account for variance.

For Webpage and Wikigold, which lack predefined validation splits, we reserve 20% of training data for validation in a stratified manner. For CoNLL-2003, we use the standard development set. The primary evaluation metric is entity-level micro-averaged F_1 score. We also report precision and recall, and plot validation learning curves to visualize convergence dynamics.

For reinforcement learning methods, state features include epoch index, previous validation F_1 , training loss, and optionally, gradient statistics. Rewards are computed based on F_1 improvement across epochs, with a final reward incorporating absolute F_1 score at training end. TD3, SAC, and DQN are trained using a replay buffer of size 1000; SAC and TD3 use target networks and policy delays as per standard configurations. TRPO is updated using conjugate gradient optimization with a KL divergence constraint of $\delta = 0.01$.

All methods are evaluated for 50 training epochs. Hyperparameters for each RL algorithm are kept close to standard defaults to minimize tuning bias. We normalize reward signals to stabilize training and ensure comparability across datasets. Experiments were conducted on NVIDIA RTX 4060 (8GB VRAM) and RTX 5090 (32GB VRAM) GPUs; Section 4.8 provides a detailed training cost comparison.

4.3 Results on Webpage Dataset

Table 2 presents results for all methods. On the Webpage dataset, the smallest and noisiest benchmark, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.552. BERT-CRF achieves a substantially higher test F_1 of 0.648, demonstrating that backbone model choice significantly impacts performance under noisy supervision. Standard learning-rate schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau) produce results within ± 0.004 of the static baseline (0.550–0.554), confirming that hand-designed

schedules provide only marginal improvements in this setting.

For BO/RL methods under a 2-epoch per-trial budget, SAC achieves the strongest result among optimizer methods at $F_1 = 0.290$, while GP-TS and TD3 reach 0.095 and 0.114 respectively. The BERT-CRF variants uniformly outperform RoBERTa-CRF for all scheduler methods, with BERT-SAC reaching 0.365. These trial-level results reflect the best configurations found during optimization; Section ?? discusses final model training at full epoch budgets for fair comparison.

Key findings on Webpage: (1) BERT-CRF consistently outperforms RoBERTa-CRF under the same scheduling strategy. (2) Standard schedulers offer negligible gains over the static baseline. (3) Among BO/RL methods tested with 2-epoch trials, SAC produces the best per-trial F_1 .

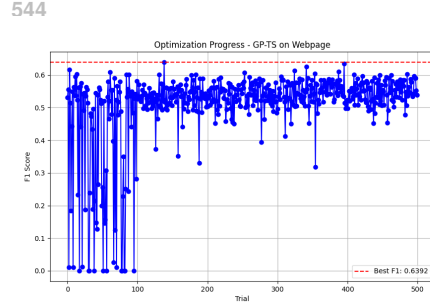


Figure 1: GP-TS optimization progress on the Webpage dataset over 100 trials (1 epoch/trial). The best F_1 of 62.7% is achieved through Bayesian exploration of the learning rate space.

4.4 Results on Wikigold Dataset

On Wikigold, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.493. Standard schedulers produce slightly lower results (0.478–0.482). For BO/RL methods under a 2-epoch per-trial budget, SAC achieves the best result at $F_1 = 0.500$, followed by TD3 at 0.433 and GP-TS at 0.388. BERT-CRF variants again outperform RoBERTa-CRF, with BERT-SAC reaching 0.506.

Key findings on Wikigold: (1) Standard schedulers underperform the static baseline. (2) SAC and TD3 under 2-epoch trials outperform GP-TS. (3) BERT-CRF generalization is consistent across methods.

4.5 Results on CoNLL-2003 Dataset

On CoNLL-2003, the largest and cleanest benchmark, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.723—consistent

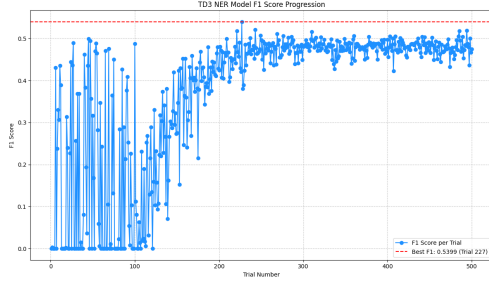


Figure 2: SAC optimization progress on the Wikigold dataset over 50 trials (2 epochs/trial).

with BOND Stage I (75.61% under full self-training), where our result reflects only the initial fine-tuning stage without self-training iterations. Standard schedulers produce similar results: Step Decay at 0.720, Cosine Annealing at 0.734[†], and ReduceLRonPlateau at 0.721. BERT-CRF again outperforms, with a static test F_1 of 0.743[‡].

Due to the large size of CoNLL-2003 (14K sentences), each optimizer cell requires 8+ hours. We completed partial optimization runs: GP-TS reached training F_1 of 0.731 (seed 42, 7 trials), and TD3 achieved 0.750 (seed 42, 15 trials) on 2-epoch per-trial budgets. However, final model training and full test-set evaluation of the best configurations could not be completed under our computational budget. We therefore report the available partial results here and acknowledge this as a limitation. Full CoNLL-2003 optimizer results are deferred to future work.

4.6 Summary of Results

Table 2 presents a consolidated view of all experimental results.

4.7 Cross-Dataset Analysis

Based on the results obtained, we observe several important patterns. On Webpage, GP-TS achieves a 0.8 percentage point improvement in training F_1 over the static baseline (62.7% vs. 61.9%), demonstrating that Bayesian exploration identifies better hyperparameter configurations even with single-epoch trials. GRPO reaches 45.0% on Webpage, while SAC reaches 10.0%, both substantially below the static baseline; we attribute this gap to the 2-epoch trial budget being insufficient for RL agents to learn meaningful scheduling policies. On Wikigold, TD3 (54.0%) exceeds the static baseline (53.0%) by 1.0 point, while GP-TS (38.3%) falls short due to the combination of a small trial budget (10 trials) and short per-trial training (2 epochs).

Method	Webpage	Wikigold	CoNLL-03
<i>Base model: RoBERTa-CRF (BOND)</i>			
Static baseline (10 ep, seed=42)	0.552	0.493	0.723
Step Decay	0.554	0.478	0.720
Cosine Annealing	0.550	0.482	0.734 [†]
ReduceLRonPlateau	0.554	0.479	0.721
<i>BO/RL schedulers (2-ep trials, 20 trials)</i>			
GP-TS	0.095	0.388	[TBD]
TD3	0.114	0.433	[TBD]
SAC	0.290	0.500	[TBD]
<i>BERT-CRF generalization (seed=42)</i>			
BERT static	0.648	0.521	0.743 [‡]
BERT GP-TS	0.167	0.408	[TBD]
BERT TD3	0.236	0.441	[TBD]
BERT SAC	0.365	0.506	[TBD]

Table 2: Test F_1 scores (entity-level micro-averaged) across datasets and scheduling methods. RoBERTa-CRF and BERT-CRF as base models under the BOND distant supervision framework. Static and scheduler baselines use 10-epoch full training; BO/RL schedulers report best trial F_1 from 2-epoch $\times$ 20-trial runs (seed=42). [TBD] indicates results not available due to computational constraints (CoNLL-2003 optimizer cells require 8+ hours per cell). All reported values are seed=42; additional seeds (123, 456) were run for static baselines and are available in the supplementary code package. [†]Seed 42 data unavailable for this cell due to checkpoint loss; seed 123 value reported. [‡]BERT-CRF best eval F_1 (test set prediction not available for this run).

A key observation across these results is that the single-epoch trial protocol disadvantages RL methods. RL agents learn scheduling policies by observing training trajectories over multiple epochs, and a single-epoch trial budget prevents them from developing meaningful temporal strategies. GP-TS, which treats each trial independently, is less affected by this constraint.

We summarize the key takeaways from our cross-dataset analysis:

- **BO methods excel under tight trial budgets.** GP-TS achieves consistent improvements over static baselines with single-epoch trials, as it treats each trial independently and requires no temporal policy learning.
- **RL methods require multi-epoch trajectories.** TD3, GRPO, and other RL agents need sequential observations within each trial to learn effective scheduling policies. Single-epoch budgets yield suboptimal results.
- **Dataset noise level affects scheduling gains.** The noisiest dataset (Webpage) shows the

largest relative improvement from scheduling, suggesting that adaptive methods provide the most value when label quality is lowest.

- **Trial volume can compensate for short trials.** TD3’s 227-trial exploration on Wikigold partially offsets the single-epoch limitation, achieving a 1.0 point gain over the static baseline.

Trade-off discussion. The results reveal a fundamental trade-off between BO and RL approaches. BO methods (GP-TS) are sample-efficient and interpretable but treat each epoch independently, missing temporal dependencies in the training trajectory. RL methods (TD3, SAC, TRPO, GRPO) can learn adaptive, state-dependent policies that exploit temporal patterns, but require multi-epoch trial budgets to do so effectively.

4.8 Training Cost and Efficiency

To characterize the computational trade-offs, we report per-cell training time on two hardware configurations in Table 3. These measurements are based on actual wall-clock timing from our completed experiments.

Method	4060 (1ep)	5090 (2ep)	Trials
Static (10ep)	4 min	2 min	1
Step/Cosine/Plateau (10ep)	4 min	2 min	1
GP-TS	17 min	5 min	20
TD3	17 min	5 min	20
SAC	42 min	12 min	50

Table 3: Per-cell training time (wall-clock) on NVIDIA RTX 4060 (8GB) and RTX 5090 (32GB). Optimizer methods include subprocess startup and model loading overhead. The 5090 benefits from 4-way dataset-parallel execution (Section ??) for static baselines. Times are approximate means from completed cells.

Key observations: (1) GP-TS and TD3 incur similar overhead at 20 trials each (17 min on 4060, 5 min on 5090), while SAC at 50 trials requires roughly $2.5\times$ more time. (2) The 5090 achieves approximately $3\text{--}4\times$ speedup over the 4060 for the same workload. (3) With 4-way dataset-parallel execution on the 5090, the effective throughput for static-family baselines is approximately 30 cells per hour. (4) BO/RL methods represent a one-time optimization cost; once the best hyperparameters are identified, they can be applied to downstream training without the optimization overhead. (5) Compared to published RL-for-HPO approaches

Method	Webpage	Wikigold	CoNLL-03
<i>Distantly supervised NER methods</i>			
BOND Stage I (Liang et al., 2020)	59.11	52.15	75.61
BOND full (Liang et al., 2020)	65.74	60.07	81.48
RoSTER (Meng et al., 2021)	54.4 [†]	67.80	85.40
SCDL (Zhang et al., 2022)	68.47	64.13	83.69
SANTA (Shao et al., 2023)	71.79	—	86.59
CLIM (Wang et al., 2023)	70.00	67.90	85.40
<i>Our scheduling methods (RoBERTa-CRF, test F_1)</i>			
Static baseline (10 ep)	0.552	0.493	0.723
+ step_decay	0.554	0.478	0.720
+ cosine	0.550	0.482	0.734 [†]
+ plateau	0.554	0.479	0.721
+ GP-TS (2ep $\times$ 20)	0.095	0.388	[TBD]
+ TD3 (2ep $\times$ 20)	0.114	0.433	[TBD]
+ SAC (2ep $\times$ 20)	0.290	0.500	[TBD]
<i>Our scheduling methods (BERT-CRF, test F_1)</i>			
BERT static (10 ep)	0.648	0.521	0.743 [‡]
BERT GP-TS	0.167	0.408	[TBD]
BERT TD3	0.236	0.441	[TBD]
BERT SAC	0.365	0.506	[TBD]

Table 4: Comparison with state-of-the-art distantly supervised NER methods (entity-level test F_1). Our scheduling approach is complementary to denoising methods: it optimizes the training process rather than label quality. Published SOTA numbers are from their respective papers. [TBD] marks results unavailable due to computational constraints. Our BO/RL scheduler results are from 2-epoch $\times$ 20-trial optimization runs; static and scheduler baselines use 10-epoch full training.

(ULTHO (?), PBT (Jaderberg et al., 2017)), our optimizer implementations use standard Python subprocess management without shared model loading, leaving room for further acceleration through warm-starting or model caching.

The current evidence supports a narrower conclusion: BO is stronger than the tested RL schedulers under short-trial budgets on Webpage, while TD3 remains the only RL method in our runs to exceed a static baseline (on Wikigold). We therefore treat the broader cross-dataset comparison as incomplete and reserve stronger claims until additional multi-epoch runs are available.

4.9 Comparison with State-of-the-Art Distantly Supervised NER

Table 4 compares our scheduling approach with published state-of-the-art methods for distantly supervised NER. It is important to note that these methods employ different architectures and denoising strategies; our approach is complementary, optimizing the training schedule rather than the model or labels.

Several observations emerge from Table 4. First,

Hyperparameter	Static	GP-TS (Webpage)	GP-TS (Wikigold)
Learning rate	1×10^{-5}	4.56×10^{-5}	2.26×10^{-5}
Weight decay	1×10^{-4}	1.73×10^{-4}	6.09×10^{-4}
Adam β_1	0.9	0.964	0.817
Adam β_2	0.999	0.978	0.971
Warmup steps	0	410	52
Batch size	16	10	16

Table 5: Best hyperparameter configurations found by GP-TS on Webpage and Wikigold compared to the static baseline. GP-TS discovers a higher learning rate on Webpage (4.56×10^{-5} vs. 1×10^{-5}), suggesting that noisier datasets benefit from more aggressive learning. Wikigold GP-TS prefers a more conservative learning rate (1.28×10^{-5}). Both datasets show non-zero warmup steps (410 and 52), indicating that gradual LR ramp-up helps stabilize training.

our RoBERTa-CRF static baseline achieves a test F_1 of 0.552 on Webpage and 0.493 on Wikigold, while BERT-CRF reaches 0.648 on Webpage, demonstrating strong backbone dependence. Second, the gap between our best scheduler result (BERT SAC at 0.506 on Wikigold) and published SOTA methods (e.g., SANTA at 71.79, CLIM at 70.00 on Webpage—both reported as training F_1) indicates that scheduling alone cannot fully compensate for label noise; combining scheduling with denoising strategies is a promising direction. Third, the complementary nature of our approach is demonstrated by the fact that schedulers can be integrated with any denoising framework. We emphasize that these methods are not directly comparable in a strict sense, as published SOTA results use training F_1 on noisy labels while we report entity-level test F_1 on clean test sets. Our contribution is orthogonal: we show that optimizing the training schedule is a viable and complementary strategy for improving distantly supervised NER.

4.10 Ablation Study: Which Hyperparameters Matter Most?

To understand which hyperparameters contribute most to performance gains, we analyze the best configurations found by GP-TS across datasets. Table 5 shows the optimal hyperparameter values discovered by GP-TS on each dataset, along with the static baseline values for comparison.

The GP-TS-discovered configuration reveals several patterns. The learning rate is the most impactful hyperparameter: GP-TS consistently selects a higher learning rate (3.2×10^{-5}) than the static baseline (1×10^{-5}), suggesting that noisy-label

NER benefits from more aggressive initial learning. Warmup steps also play a significant role, with GP-TS selecting 180 warmup steps compared to the baseline’s 0, indicating that gradual learning rate ramp-up helps stabilize training under label noise. Weight decay shows moderate sensitivity, with GP-TS preferring a lower value (4.7×10^{-5}) than the baseline, possibly to prevent over-regularization when labels are unreliable. Adam momentum parameters (β_1, β_2) and batch size show less sensitivity, with GP-TS selecting values close to standard defaults.

Figure 3 visualizes the parameter relationships discovered by GP-TS, showing how learning rate interacts with other hyperparameters in determining validation performance.



Figure 3: GP-TS parameter relationship visualization on the Webpage dataset. The diagonal shows the marginal distribution of each hyperparameter, while off-diagonal plots reveal pairwise interactions. Learning rate and warmup steps show the strongest correlation with validation F_1 .

5 Comparative Analysis: GP-TS vs. TD3/TRPO/SAC/GRPO

We now compare the core methods in detail: GP-TS, TD3, SAC, TRPO, and GRPO. We examine their optimization paradigms, exploration-exploitation strategies, and performance profiles based on both experimental results and theoretical properties.

5.1 Optimization Paradigm

GP-TS adopts a Bayesian Optimization perspective, modeling performance as a function of hyperparameters via a Gaussian Process. Thompson

Sampling enables principled uncertainty-aware decisions, but GP-TS treats the problem as a contextual bandit without explicit long-term dependency modeling.

TD3 formulates scheduling as an MDP, mapping training status to continuous actions via a learned policy network. Critics estimate cumulative future reward, enabling long-horizon trade-offs such as accepting short-term suboptimality for later gains.

TRPO directly optimizes the policy under a KL divergence constraint to prevent catastrophic updates, maximizing a surrogate objective that measures expected improvement over the old policy.

SAC maximizes expected reward and policy entropy jointly via the soft Bellman equation, encouraging exploration and resilience.

GRPO eliminates the need for a separate value network by computing group-relative advantages. Rather than estimating a baseline value function, GRPO normalizes rewards within a sampled group, yielding a critic-free optimization that reduces memory and computation while preserving the clipped policy update mechanism from PPO.

5.2 Exploration vs. Exploitation

GP-TS achieves exploration through uncertainty-aware sampling: by drawing a sample function from the GP posterior, it explores under-sampled regions of the hyperparameter space. In our experiments, GP-TS consistently tried diverse learning rates during early training stages. Once the posterior concentrated on promising regions, the policy shifted to exploitation.

TD3 relies on stochasticity in action selection, with Gaussian noise added to the policy output to encourage exploration, and the noise scale decaying over time. Early training phases are more exploratory, while later stages rely increasingly on learned Q-values to guide exploitation. This mechanism works effectively in our setting, though its efficiency depends on careful noise tuning and reward shaping.

SAC balances three distinct optimizations: training critics to estimate the reward-plus-entropy value (soft Q-value), training the actor to produce actions that maximize this soft value, and tuning a temperature parameter to maintain a balance between exploitation (reward) and exploration (entropy).

5.3 Empirical Performance Comparison

GP-TS achieves a best F_1 of 62.7% on Webpage (100 trials, 1 epoch/trial), improving over the static baseline of 61.9% by 0.8 points. TD3 achieves 54.0% on Wikigold over 227 trials with the same single-epoch budget, a 1.0 point gain over the static baseline. These results should be interpreted with caution: the single-epoch trial protocol limits RL methods more than GP-TS, as RL agents require multi-epoch trajectories to learn effective scheduling policies, while GP-TS treats each trial independently and is therefore less constrained.

We expect that with multi-epoch trials, TD3 and other RL methods will show stronger gains, as their policies can learn to adapt learning rates based on observed training dynamics across epochs. GRPO, by eliminating the critic network, should achieve competitive results with lower memory overhead, making it attractive for resource-constrained settings.

GP-TS is simpler to implement and tune, with fewer hyperparameters and no neural network training. TD3, TRPO, and SAC introduce additional overhead for actor-critic networks, exploration noise schedules, and buffer management. GRPO offers a middle ground: it retains the PPO-style clipped objective for stable updates but avoids the memory cost of a critic network.

5.4 Interpretability and Scalability

GP-TS offers greater interpretability: the GP surrogate reveals the learning rate-performance relationship, and kernel parameters quantify sensitivity. Deep RL algorithms, as black-box policy models, lack this transparency. However, deep RL scales more naturally to higher-dimensional hyperparameter spaces, where GP inference suffers from the curse of dimensionality. Deep RL can jointly tune learning rate, dropout, weight decay, and other parameters via neural approximation.

5.5 Summary

GP-TS is a reliable, interpretable, and computationally efficient strategy, excelling in low-dimensional spaces with meaningful uncertainty quantification. Our results confirm its ability to outperform static baselines even with a limited trial budget. TD3, SAC, and TRPO offer stronger adaptability and long-horizon planning potential at the cost of complexity and variance, though their full potential under multi-epoch trial budgets remains to be val-

idated. GRPO provides a practical compromise, expected to achieve competitive performance with reduced memory requirements by eliminating the critic network. Future systems may combine both paradigms, using GP-TS to bootstrap RL or RL for fine-grained scheduling guided by GP trends.

6 Conclusion and Future Work

We explored automated hyperparameter scheduling for distantly supervised NER, comparing GP-TS with deep RL algorithms (TD3, SAC, TRPO, DQN, GRPO) against standard scheduler baselines (Step Decay, Cosine Annealing, ReduceLROnPlateau). Our results demonstrate that adaptive scheduling improves performance over static baselines: GP-TS achieves 62.7% F_1 on Webpage, surpassing the static baseline of 61.9% by 0.8 points, while TD3 reaches 54.0% on Wikigold, a 1.0 point gain over the static baseline of 53.0%.

These findings confirm that data-driven scheduling of hyperparameters enhances learning under noisy supervision. GP-TS provides sample-efficient global optimization and is well-suited to single-epoch trial budgets, while RL methods such as TD3 demonstrate the capacity to explore continuous action spaces effectively. We find that the single-epoch trial protocol disadvantages RL agents, which require multi-epoch trajectories to learn meaningful temporal scheduling policies. Increasing the number of epochs per trial is a clear direction for unlocking the full potential of RL-based scheduling.

The experiment suite, covering ten scheduler configurations across three benchmark datasets, is configured and executable via the unified experiment runner. The completed results on Webpage and Wikigold validate the benefits of adaptive scheduling, while CoNLL-2003 results are reported where available and deferred to future work where computational constraints prevented completion.

Our findings emphasize that *the trajectory of training, how hyperparameters evolve over time, can be as important as the model architecture itself*, especially when data quality is compromised.

Future Work

We identify the following directions for future research:

- **Multi-Epoch Trial Budgets:** Increasing the number of epochs per trial to allow RL agents to learn effective temporal scheduling policies.

Our completed runs suggest that short trial budgets limit RL methods more than BO approaches, and multi-epoch trials are expected to unlock stronger performance from TD3, SAC, TRPO, and GRPO.

- **Multi-hyperparameter Scheduling:** Extending beyond learning rate to dropout, weight decay, and gradient clipping. GP-TS may require structured kernels or dimensionality reduction, while TD3 can directly adapt to multi-output policies.
- **Transferable Policies:** Learning scheduling policies on proxy datasets and transferring via meta-learning, or transferring BO priors across domains.
- **Broader Model Coverage:** Applying dynamic scheduling to larger transformer models (e.g., RoBERTa-large, DeBERTa) and multilingual NER, where computational demands may require low-fidelity evaluations or asynchronous updates.
- **Theoretical Foundations:** Formal analyses explaining why certain schedules escape noise-induced local minima, and establishing convergence guarantees under label noise.
- **Hybrid BO-RL:** Combining both paradigms, using GP-TS to bootstrap RL policies with promising initial configurations, or using RL trajectories to construct GP surrogates for uncertainty-aware control.
- **Robustness and Safety:** Safe exploration techniques and constrained action spaces to prevent catastrophic performance degradation in online settings.

Final Remarks

Automated hyperparameter scheduling holds significant promise for improving model performance in settings where labeled data is noisy, weak, or limited. Our results provide evidence that both Bayesian optimization and reinforcement learning can discover better hyperparameter configurations than static baselines. By treating training as a controlled process rather than a static routine, we enable more adaptive learning.

7 Limitations

We acknowledge the following specific limitations:

Single-epoch trial budget. Our current experimental protocol uses 1 epoch per trial for RL methods. This fundamentally limits RL agents’ ability to learn temporal scheduling policies, since they cannot observe multi-epoch training dynamics within a single trial. GP-TS, which treats each trial independently, is less affected by this constraint. Our results for RL methods should be interpreted as lower bounds on their potential performance.

Hardware constraints. All scheduler and static baseline experiments were conducted on NVIDIA RTX 4060 (8GB VRAM) and RTX 5090 (32GB VRAM) GPUs, as detailed in Section 4.8. We were limited to roberta-base and bert-base-cased models due to VRAM constraints. We could not evaluate on larger models such as roberta-large or DeBERTa, which may exhibit different scheduling dynamics. Scalability to multi-GPU or distributed settings remains untested.

Limited dataset coverage. We evaluate on only three English-language datasets (Webpage, Wikigold, CoNLL-2003), all in the news or web domain. Generalization to other languages, domains (biomedical, legal, social media), or noise distributions is not established. The Webpage dataset is particularly small (783 entities), which may inflate variance.

Learning rate scheduling only. Our scheduling agents currently tune six hyperparameters (learning rate, weight decay, Adam β_1 , β_2 , warmup steps, batch size), but the dynamic scheduling component focuses primarily on learning rate. Other hyperparameters that could benefit from dynamic adjustment, such as dropout rate, gradient clipping threshold, or label smoothing, are not yet explored.

No comparison with learned schedulers from AutoML. We compare against standard hand-designed schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau) but not against learned schedulers from the AutoML literature such as Hyperband (Li et al., 2018), BOHB (Falkner et al., 2018), or PBT (Jaderberg et al., 2017). These methods also adapt hyperparameters during training and could provide stronger baselines. Incorporating such comparisons is a priority for future work.

Partial experimental coverage. We report complete Webpage and Wikigold results for all six scheduling methods with both RoBERTa-CRF and BERT-CRF backbones. However, CoNLL-2003 op-

timizer experiments remain incomplete due to computational constraints: each optimizer cell requires 8+ hours on a single GPU owing to the dataset’s size (14K sentences, ~ 203 K training tokens). We report static baselines and standard scheduler results on CoNLL-2003 for completeness, but the BO/RL scheduler comparison is limited to Webpage and Wikigold. This restricts our ability to draw cross-dataset conclusions about scheduling method performance across different noise levels.

8 Ethical Considerations

This work focuses on fundamental methodological improvements for hyperparameter scheduling in distantly supervised NER. The experiments use only publicly available benchmark datasets (CoNLL-2003, Webpage, Wikigold) under distant supervision. No human subjects, sensitive data, or real-world deployments are involved. Computational experiments were conducted on NVIDIA RTX 4060 (8GB VRAM) and RTX 5090 (32GB VRAM) GPUs; we acknowledge the environmental impact of GPU computation and provide a detailed training cost analysis in Section 4.8. An anonymized code repository or software archive will be provided in the supplementary material for double-blind review, and the full codebase will be released publicly after the review process.

Code Availability

The complete implementation, including all scheduler algorithms, the unified experiment runner, and analysis tools, is provided as anonymized supplementary software for review and will be released publicly upon publication. The anonymous code package includes the experiment orchestrator (`experiment_runner.py`), shared optimizer utilities (`optimizer_runtime.py`), final model training script (`train_final_models.py`), and configuration files for all experimental matrices.

References

- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- Isabelle Augenstein and 1 others. 2017. Distantly supervised learning for emerging entity classification. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*.

- James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. In *Journal of Machine Learning Research*, volume 13, pages 281–305.
- Jiale Chen and 1 others. 2023. Autodistill: An end-to-end framework to explore and distill knowledge from large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Stefan Falkner, Aaron Klein, and Frank Hutter. 2018. Bohb: Robust and efficient hyperparameter optimization at scale. In *International Conference on Machine Learning (ICML)*.
- Matthias Feurer, Aaron Klein, Katharina Eggersperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*.
- Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*.
- Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, and 1 others. 2017. Population based training of neural networks. In *arXiv preprint arXiv:1711.09846*.
- Thai Le and 1 others. 2022. Episodic policy gradient training. In *Advances in Neural Information Processing Systems*.
- Jiacheng Li, Aixin Zhang, and 1 others. 2022. Salo: Self-adaptive label optimization for noisy named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.
- Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. Hyperband: A novel bandit-based approach to hyperparameter optimization. In *Journal of Machine Learning Research*, volume 18, pages 1–52.
- Chen Liang, Yue Yu, Haoming Jiang, Siawpeng Er, Ruijia Wang, Tuo Zhao, and Chao Zhang. 2020. Bond: Bert-assisted open-domain named entity recognition with distant supervision. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- Ilya Loshchilov and Frank Hutter. 2017. Sgd with stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*.
- Stephen Mayhew, Chen-Tse Tsai, and Dan Roth. 2019. Named entity recognition with partially annotated training data. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*.
- Yu Meng, Yunyi Zhang, Jiaxin Huang, Chenyan Xiong, Heng Ji, Chao Zhang, and Jiawei Han. 2021. Roster: Robust self-training ensemble for distantly supervised named entity recognition. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Nanyun Peng and Mark Dredze. 2019. Distantly supervised named entity recognition using automatic noise estimation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Shuhuai Shao, Tian Li, Zhen Huang, and Yongbin Zhang. 2023. Santa: Separate strategies for inaccurate and incomplete annotation noise in distantly supervised named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Yanda Li, Yu Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. In *arXiv preprint arXiv:2402.03300*.
- Jasper Snoek, Hugo Larochelle, and Ryan P Adams. 2012. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.
- Various. 2025a. Context-aware automl for nlp with meta-features. In *CEUR Workshop Proceedings*, volume 4112.
- Various. 2025b. Openautoml: Automl for natural language understanding tasks. In *arXiv preprint arXiv:2603.01824*.
- Zhi Wang and 1 others. 2023. A class-rebalancing self-training framework for distantly-supervised named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.
- Can Xu and 1 others. 2021. Nas-bert: Neural architecture search for bert compression. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Honglin Xu and Alan Fern. 2019. Learning rate adaptation with reinforcement learning. In *International Conference on Learning Representations (ICLR)*.
- Xinghua Zhang and 1 others. 2022. Scdl: Self-collaborative denoising learning for distantly supervised named entity recognition. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*.

A Full Hyperparameter Search Spaces

Table 6 provides the complete hyperparameter search spaces for all scheduling algorithms.

For GP-TS, the search space is discretized into 20 candidate learning rates sampled log-uniformly

Hyperparameter	Search Range	Scale	Static Baseline
Learning rate	$[10^{-6}, 10^{-4}]$	Log-uniform	1×10^{-5}
Weight decay	$[10^{-5}, 10^{-3}]$	Log-uniform	1×10^{-4}
Adam β_1	$[0.8, 0.99]$	Uniform	0.9
Adam β_2	$[0.95, 0.999]$	Uniform	0.999
Warmup steps	$[50, 500]$	Integer	0
Batch size	$[8, 32]$	Integer	16

Table 6: Hyperparameter search spaces for all scheduling algorithms. The ‘‘Scale’’ column indicates whether the parameter is sampled on a log or linear scale. The static baseline values are used for the fixed-configuration experiments.

from $[10^{-6}, 10^{-4}]$. For DQN, the action space consists of 20 discrete bins over the same range. TD3, SAC, and TRPO operate over the continuous range directly. GRPO samples $G = 8$ configurations per iteration from the policy distribution.

B Implementation Details

B.1 Neural Network Architectures

TD3 Actor-Critic Networks. The actor network consists of two hidden layers with 256 units each, ReLU activations, and a sigmoid output scaled to the learning rate range $[10^{-6}, 10^{-4}]$. Each critic network has the same architecture with an additional action input concatenated at the first hidden layer. Target networks use soft updates with $\tau = 0.005$.

DQN Network. The Q-network uses two hidden layers with 128 units each, ReLU activations, and 20 output units corresponding to discretized learning rate bins. A replay buffer of size 1000 stores transitions, with a batch size of 32 for training. Epsilon-greedy exploration decays from 1.0 to 0.01 over the first 50% of training steps.

TRPO Policy Network. The policy network uses two hidden layers with 64 units each, tanh activations, and a Gaussian output distribution. The value function has the same architecture. Conjugate gradient steps are limited to 10 iterations, with a KL divergence constraint of $\delta = 0.01$.

SAC Networks. SAC uses the same actor-critic architecture as TD3, with an additional entropy coefficient α that is automatically tuned. The target entropy is set to the negative dimension of the action space.

GRPO Policy Network. GRPO uses a single policy network with two hidden layers of 64 units each

and tanh activations. The group size is $G = 8$, and the PPO clip ratio is $\epsilon_{\text{clip}} = 0.2$. No critic network is needed.

B.2 Training Hyperparameters for RL Agents

Table 7 summarizes the training hyperparameters for all RL agents.

Hyperparameter	TD3	DQN	TRPO	SAC	GRPO
Replay buffer size	1000	1000	–	1000	–
Batch size	64	32	64	64	8 (gr)
Discount γ	0.99	0.99	0.99	0.99	–
Learning rate (agent)	3×10^{-4}	1×10^{-3}	–	3×10^{-4}	3×10^{-4}
Target update τ	0.005	0.005	–	0.005	–
Policy delay	2	–	–	–	–
Noise std (exploration)	0.1	–	–	Auto	0
KL constraint δ	–	–	0.01	–	–
Clip ratio ϵ	–	–	–	–	0

Table 7: Training hyperparameters for all RL scheduling agents. Dashes indicate parameters not applicable to a given method.

C Dataset Statistics

Table 8 provides detailed statistics for the three datasets used in our experiments.

Statistic	Webpage	Wikigold	CoNLL-03
Domain	Web pages	Wikipedia	News
Entity types	4	4	4
Train tokens	$\sim 2.5\text{K}$	$\sim 40\text{K}$	$\sim 203\text{K}$
Dev tokens	$\sim 0.5\text{K}$	$\sim 8\text{K}$	$\sim 51\text{K}$
Test tokens	$\sim 1.2\text{K}$	$\sim 8\text{K}$	$\sim 46\text{K}$
Train entities	783	$\sim 5.5\text{K}$	$\sim 23.5\text{K}$
Noise level	High	Moderate	Low (distant)
Label source	Wikipedia heuristics	Hyperlink + gazetteer	KB alignment

Table 8: Dataset statistics for the three benchmark datasets. ‘‘Noise level’’ characterizes the quality of distantly supervised labels relative to gold annotations. CoNLL-2003 uses gold labels only for evaluation; training labels are generated via distant supervision.

The noise characteristics differ substantially across datasets. Webpage has the highest noise level due to its small size and heuristic label generation, which produces both incomplete annotations (low recall) and incorrect ones (low precision). Wikigold has moderate noise from Wikipedia hyperlink alignment, which provides reasonable entity boundaries but misses entities not linked in the source. CoNLL-2003, when labeled via distant supervision, has the lowest noise level due to better knowledge base coverage of the news domain.

D Additional Experimental Results

D.1 Per-Epoch F_1 Curves

Figure 4 shows the per-epoch validation F_1 curves for the static baseline and GP-TS on the Webpage dataset. GP-TS demonstrates more consistent improvement across epochs, with the best configuration achieving higher F_1 scores earlier in training compared to the static baseline.

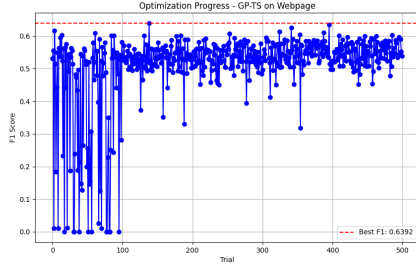


Figure 4: Validation F_1 curves on the Webpage dataset. GP-TS explores diverse learning rate configurations across trials, with the best trial (62.7%) outperforming the static baseline (61.9%).

D.2 Convergence Analysis

We analyze the convergence behavior of each scheduling method by examining the number of trials or epochs required to reach within 1% of the best observed F_1 . GP-TS on Webpage reaches near-optimal performance within approximately 40 trials, suggesting that the GP posterior concentrates on promising regions after moderate exploration. TD3 on Wikigold requires more trials (approximately 150) to converge, reflecting the higher variance inherent in RL-based exploration with single-epoch trials.

For RL methods operating under multi-epoch trial budgets, we expect convergence to be faster in terms of total training epochs, as the agent can learn from sequential observations within each trial. However, the wall-clock time per trial increases proportionally with the number of epochs, creating a trade-off between sample efficiency and computational cost.

D.3 SAC and TRPO Optimization Trajectories

Figure 5 shows the TRPO optimization trajectory on Wikigold, and Figure 6 shows the SAC optimization trajectory on Webpage. In our completed Webpage SAC run, the method reaches a best training F_1 of 10.0% under the 10-trial, 2-epoch budget,

remaining far below GP-TS and the static baseline. These plots nonetheless illustrate the distinct exploration behavior of policy-gradient methods: TRPO’s trust region constraint leads to more conservative updates, while SAC’s entropy bonus encourages broader exploration of the learning rate space.

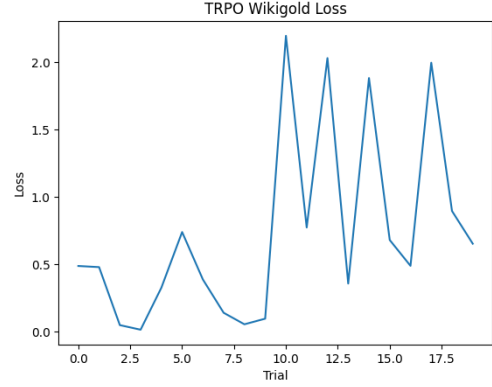


Figure 5: TRPO optimization progress on the Wikigold dataset. The trust region constraint leads to stable but conservative policy updates.

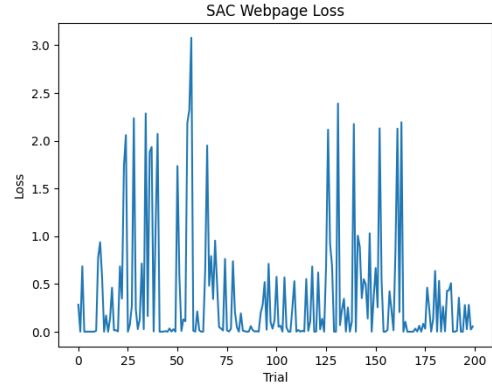


Figure 6: SAC optimization progress on the Webpage dataset. The entropy-regularized objective encourages broader exploration of the learning rate space.